Flutter'da Animasyonlar ve Geçişler

60–120 FPS hızında akıcı, duyarlı ve fizik tabanlı kullanıcı deneyimleri tasarlamak: Örtük (`Implicit`), Açık (`Explicit`), `Hero`, `PageRouteBuilder`, Kademeli (`Staggered`) ve Fizik Simülasyonları.



KURUM & DERS BİLGİSİ

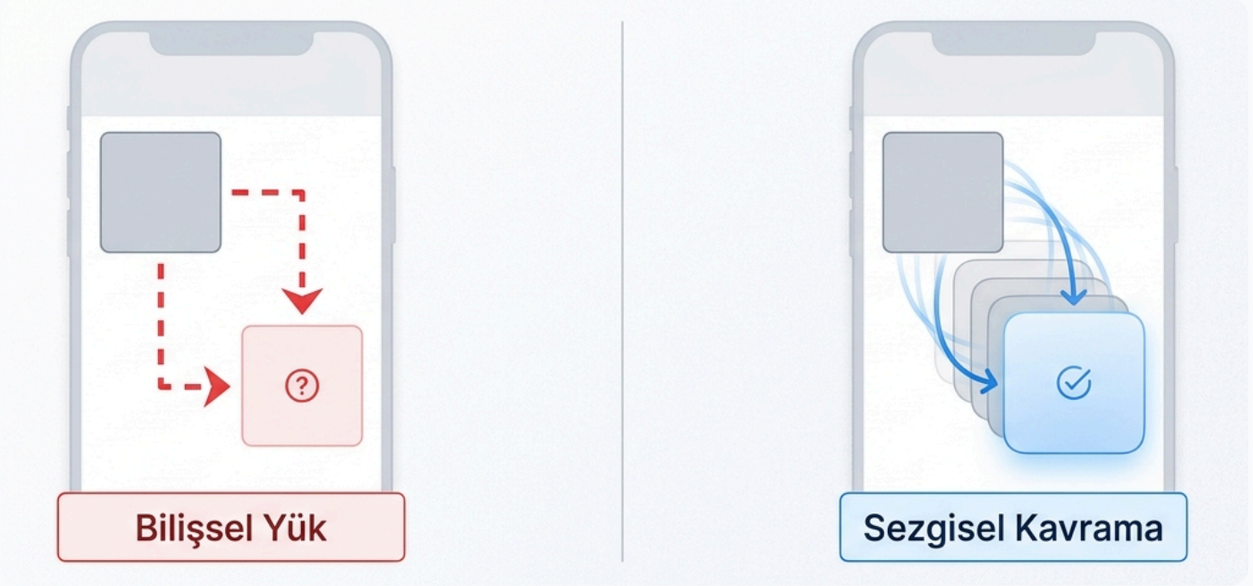
Sivas Cumhuriyet Üniversitesi
Şarkışla UBYO · Bilişim Sistemleri ve Teknolojileri
Mobil Programlama I (2026 Güz)

6. TAKIM: TEAMANİMATORS

- Aydın Yiğit Oğuz
- Arman Dündar
- Sefa Omaç
- Yusuf Cemal Durmuş

Arayüzde Hareketin Gücü: Bilişsel Yük vs. Sezgisel Kavrama

İyi tasarlanmış bir animasyon süs değil; kullanıcıya mekânsal ve nedensel bağlam sunan görsel bir rehberdir.



! Ani Kesmeler = Bilişsel Yük

Bir öğe ekranda aniden kaybolup başka bir yerde belirdiğinde, kullanıcının beyni *"Az önce ne değişti?"* sorusunu çözmek için ekstra bilişsel enerji harcar.

1 Mekânsal Süreklilik & Odak

Kareler arası yumuşak geçişler, öğenin **nereden gelip nereye gittiğini** gösterir; kullanıcının gözünü doğal biçimde yeni odak noktasına yönlendirir.

2 60–120 FPS Duyarlılık

Flutter'ın Impeller / Skia motoru her kareyi **8.3 ms – 16.6 ms** bütçesinde çizerek fiziksel dünyadaki nesneler kadar doğal ve gecikmesiz geri bildirim üretir.

Karar Ağacı: Hangi Animasyon Yaklaşımını Seçmeliyiz?

Flutter'da her senaryo için tek bir kalıp kullanılmaz; ihtiyaca en uygun soyutlama katmanı seçilir.

1. BASİT DURUM GEÇİŞİ

Örtük (Implicit) Widget'lar

Bir özelliğin (renk, boyut, opaklık) eski değerden yeni değere tek seferde kayması.

AnimatedContainer
AnimatedOpacity
AnimatedSwitcher

2. ÖZEL ÖRTÜK DEĞER

TweenAnimationBuilder

Hazır `AnimatedFoo` karşılığı olmayan bir özelliği `Controller` açmadan anime etmek.

TweenAnimationBuilder<T>
(Açı, Gölge, Blur, Clip)

3. TAM ZAMAN KONTROLÜ

Açık (Explicit) Animasyon

Döngüye giren (`repeat`), tersine dönen (`reverse`) veya çoklu adımlı koreografiler.

AnimationController
FooTransition / Interval
AnimatedBuilder

4. EKRANLAR ARASI GEÇİŞ

Hero & PageRouteBuilder

İki sayfa arasında uçan ortak görsel öğeler veya özel sayfa kayma efektleri.

Hero (tag eşleşmesi)
PageRouteBuilder
SlideTransition

5. ETKİLEŞİM & FİZİK

Fizik Simülasyonları

Sabit süre yerine kullanıcı parmak hızına, yay sertliğine ve sürtünmeye duyarlı hareket.

SpringSimulation
GravitySimulation
controller.animateWith()

> **Pratik Kural (Rule of Thumb):** Her zaman en basit soyutlama olan **Örtük (Implicit)** animasyonlarla başlayın. Yalnızca animasyonu durdurmanız, tersine çevirmeniz, tekrarlamanız veya birden çok hareketi senkronize etmeniz gerektiğinde **Açık (Explicit)** animasyonlara geçin.

Flutter Animasyon Mimarisi: 5 Temel Yapı Taşı

Flutter'da her animasyon; zaman sinyali, yönetici, eğri, değer dönüştürücü ve dinleyici widget zincirinden oluşur.

1. DONANIM SAATİ

Ticker (vsync)

Ekranın dikey yenilenme sinyaline (60/120 Hz) kilitlenir. Her yeni karede geri çağrı (`Callback`) tetikler.

➔

2. ZAMAN YÖNETİCİSİ

AnimationController

Geçen süreyi `0.0` ile `1.0` arasında doğrusal ilerleme değerine çevirir. Durumu yönetir.

➔

3. İVME & RİTM

CurvedAnimation

Doğrusal zamanı seçilen eğri fonksiyonuna (`Curves.easeOut` , `bounceOut`) göre büker.

➔

4. VERİ HARİTALAMA

Tween<T>

`0.0–1.0` oranını hedef veri tipine (`Color` , `Offset` , `double` piksel) dönüştürür.

➔

5. GÖRSEL ÇİZİM

UI Widget

`AnimatedBuilder` veya `FooTransition` yeni değeri alıp sadece ilgili düğümü çizer.

Soyut Temel Sınıf: `Animation<T>`

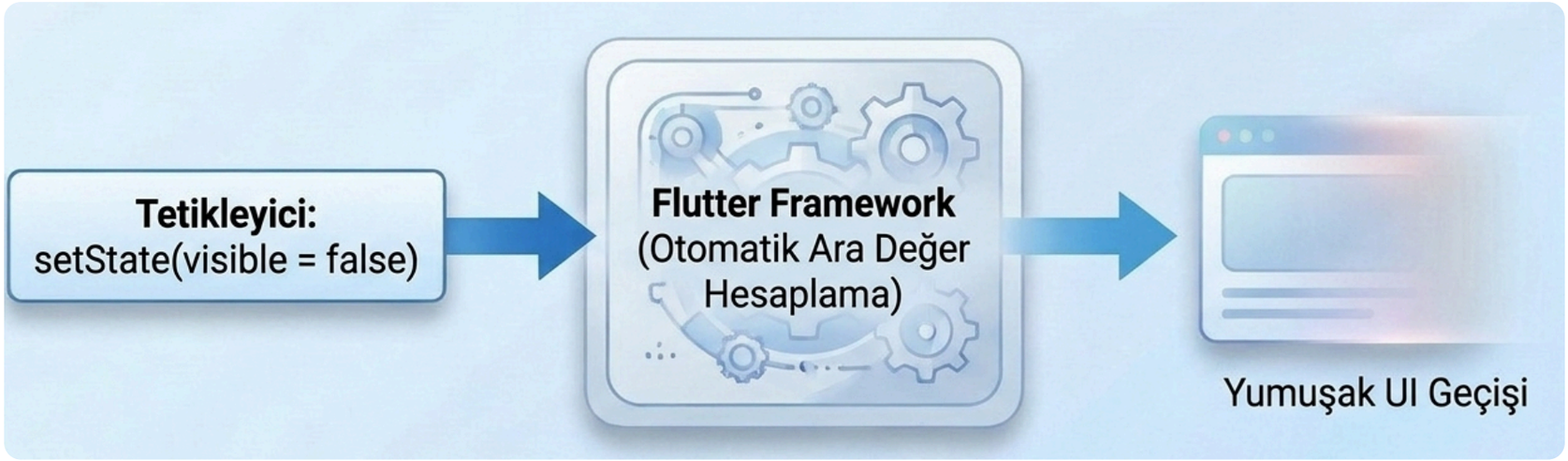
Flutter'da bir animasyon nesnesi kendi başına ekranda hiçbir şey çizmez; yalnızca **mevcut değerini (`.value`)** ve **durumunu (`.status`)** bilen, değiştiğinde dinleyicilerine (`addListener` / `addStatusListener`) haber veren reaktif bir veri kaynağıdır.

Neden Katmanlı Mimari?

Zamanlama (`Controller`), ivmelenme (`Curve`), veri tipi (`Tween`) ve görsel bileşen (`Widget`) birbirinden ayrıldığı için **tek bir** `AnimationController` aynı anda hem rengi, hem konumu, hem de boyutu farklı eğrilerle yönetebilir.

Örtük Animasyonlar: Otopilot Modu

Geliştirici yalnızca hedef (yeni) değeri bildirir; ara karelerin hesaplanması tamamen Flutter çatısına devredilir.



1 Durum (State) Değişimi

Kullanıcı etkileşimi veya veri güncellemesi sonrası `setState()` çağrılır ve widget'ın hedef özelliği (ör. `width: 100 ➔ 260`, `opacity: 1.0 ➔ 0.0`) güncellenir.

2 Dahili İnterpolasyon

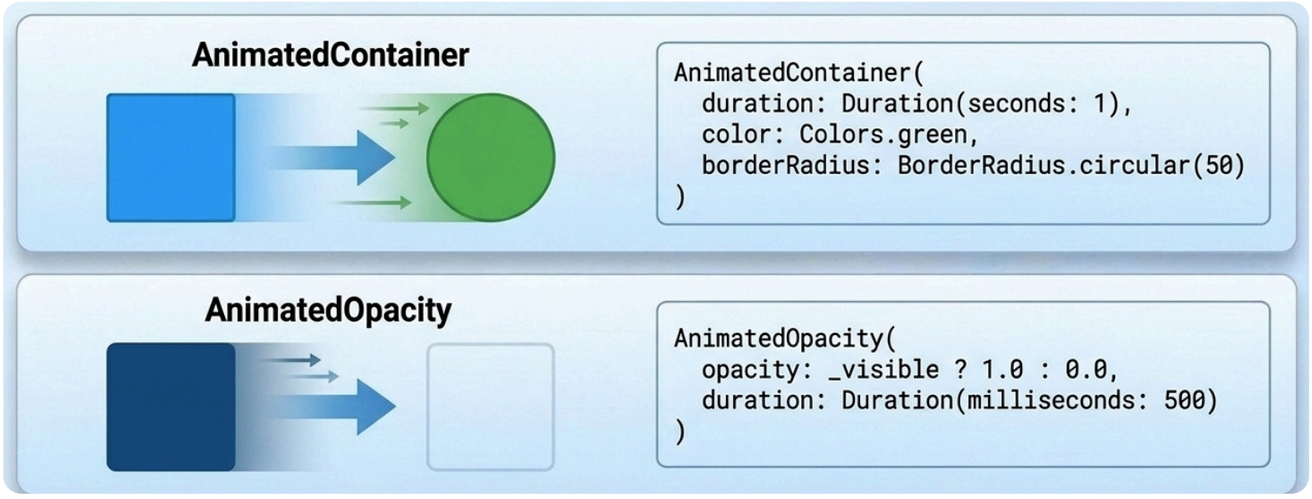
`ImplicitlyAnimatedWidget` alt sınıfı eski değer ile yeni değer arasındaki farkı algılar; kendi iç `AnimationController` ve `Tween` nesnelerini otomatik kurar.

3 Sıfır Yaşam Döngüsü Yükü

`SingleTickerProviderStateMixin` eklemeye veya `dispose()` yazmaya gerek kalmaz. Animasyon bittiğinde `onEnd` geri çağrısı ile zincirleme işlem yapılabilir.

Yaygın Örtük Widget'lar (AnimatedFoo Ailesi)

Standart widget'ların 'Animated' önekli karşılıkları, aynı parametrelerin yanı sıra duration ve curve kabul eder.



Örtük Widget	Anime Edilen Özellikler
`AnimatedContainer`	`width`, `height`, `color`, `padding`, `borderRadius`, `margin`, `alignment`, `transform`
`AnimatedOpacity`	Görünürlük (`opacity: 0.0 - 1.0`); GPU dostu saydamlık geçişi sağlar
`AnimatedAlign` / `Padding`	Hizalama (`Alignment`) ve iç boşluk (`EdgeInsets`) geçişleri
`AnimatedPositioned`	`Stack` içindeki `top`, `bottom`, `left`, `right` koordinatları
`AnimatedSwitcher`	Farklı `Key`'e sahip iki farklı alt widget arasında geçiş (Fade/Scale)

İpucu: `AnimatedContainer` içinde birden fazla özelliği (hem renk hem boyut hem kenar yuvarlaklığı) aynı `setState` içinde değiştirirseniz, hepsi eşzamanlı olarak tek bir akıcı animasyonla dönüşür!

Özel Örtük Kontrol: TweenAnimationBuilder<T>

Hazır bir AnimatedFoo widget'ı bulamadığınızda, AnimationController yazmadan özel özellikleri anime etmenin yolu.

tween_animation_builder_example.dart

```dart
TweenAnimationBuilder<double>(
  // 1. Başlangıç ve hedef değer aralığı (_targetValue değiştikçe tetiklenir)
  tween: Tween<double>(begin: 0.0, end: _targetRotation),
  duration: const Duration(milliseconds: 800),
  curve: Curves.easeOutBack,
  // 2. Sabit alt ağacı önbelleğe alarak her karede yeniden çizimi önler!
  child: const Icon(Icons.flutter_dash, size: 72, color: Colors.blue),
  // 3. Her karede güncel 'angle' değeriyle çağrılan çizim fonksiyonu
  builder: (BuildContext context, double angle, Widget? staticChild) {
    return Transform.rotate(
      angle: angle,
      child: staticChild, // Yeniden oluşturulmayan sabit widget
    );
  },
)
```

Nasıl Çalışır?

`tween.end` parametresine bağlı durum değişkeni (`_targetRotation`) her değiştiğinde, `TweenAnimationBuilder` o anki mevcut değerden yeni hedef değere doğru yumuşak bir geçiş başlatır.

Performans Sırrı: `child` Parametresi

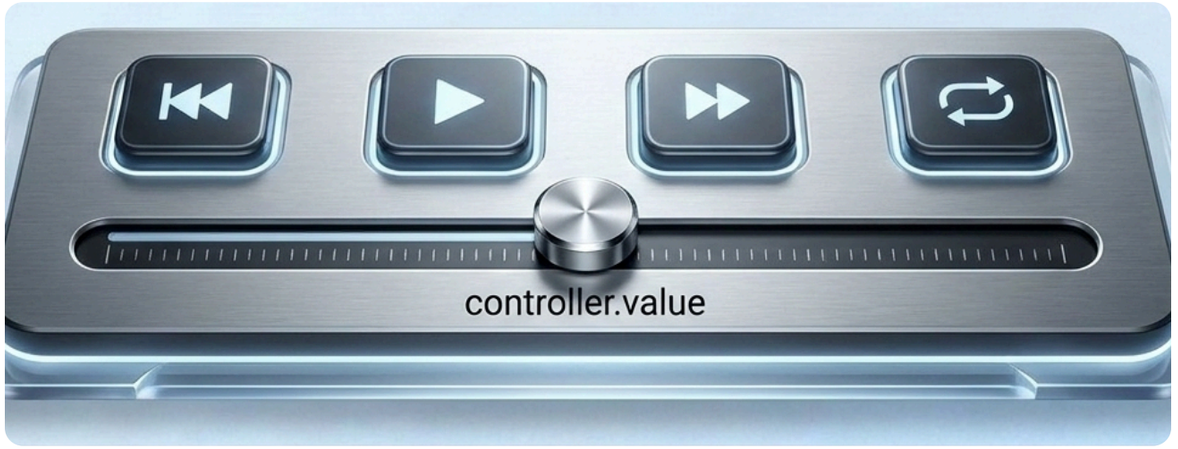
`builder` fonksiyonu saniyede 60–120 kez çalışır. Animasyondan etkilenmeyen ağır alt bileşenleri dışarıdaki `child` parametresine verirseniz, Flutter onu yalnızca **bir kez** oluşturur ve `staticChild` olarak içeri aktarır.

Ne Zaman Tercih Edilir?

`Transform.rotate` , `BackdropFilter` (bulanıklık), `CustomPaint` veya `ShaderMask` gibi özel görsel efektlerde tek yönlü geçiş yapmak için idealdir.

Açık Animasyonlar: Yönetmen Koltuğu

Zaman çizelgesini başlatmak, durdurmak, tersine sarmak veya sonsuz döngüye sokmak geliştiricinin elindedir.



`.forward()`

Animasyonu mevcut değerinden üst sınıra (`1.0`) doğru ileri yönde başlatır. Tamamlandığında durum `AnimationStatus.completed` olur.

`.reverse()`

Animasyonu mevcut konumundan başlangıç noktasına (`0.0`) doğru geriye sarar. Bittiğinde durum `AnimationStatus.dismissed` olur.

`.repeat(reverse: true)`

Animasyonu durdurulana kadar sürekli tekrarlar. `reverse: true` verilirse ileri-geri (nefes alma / nabız efekti) şeklinde salınır.

`.stop()` & `.reset()`

`stop()` animasyonu o anki değerinde dondurur; `reset()` ise anında alt sınıra (`lowerBound = 0.0`) sıfırlar.

AnimationController ve Ticker (vsync): Motorun Kalbi

Donanım yenileme hızıyla senkronize çalışan zamanlayıcı ve bellek güvenliği için zorunlu yaşam döngüsü.

explicit_controller_lifecycle.dart

```dart
class _PulseViewState extends State<PulseView>
    with SingleTickerProviderStateMixin { // 1. Ticker sağlayıcı mixin
  late final AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this, // 2. Ekran dışı kare üretimini kesen vsync kilidi
      duration: const Duration(milliseconds: 600),
      lowerBound: 0.0,
      upperBound: 1.0,
    )..repeat(reverse: true);
  }

  @override
  void dispose() {
    _controller.dispose(); // 3. KRİTİK: Ticker sızıntısını önlemek için şart!
    super.dispose();
  }
}
```

`vsync` ve `TickerProvider` Nedir?

`vsync: this` parametresi, `AnimationController`'ı cihaz ekranının dikey senkronizasyon sinyaline bağlar. Widget görünmez olduğunda (başka sekmeye geçildiğinde veya uygulama arka plana atıldığında) `Ticker` otomatik susturularak **CPU ve pil tüketimi sıfırlanır**.

Tek vs. Çoklu Controller

Sınıfta tek bir `AnimationController` varsa `SingleTickerProviderStateMixin`; birden fazla bağımsız `AnimationController` varsa `TickerProviderStateMixin` kullanılır.

Unutulan `dispose()` Hatası

Sayfa kapatıldığında `_controller.dispose()` çağrılmazsa, `Ticker` arka planda sonsuza dek çalışmaya devam eder ve Flutter konsolunda "*Ticker was not disposed*" bellek sızıntısı hatası fırlatılır.

Yerleşik Açık Widget'lar (FooTransition) ve AnimatedWidget

addListener() ve setState() kalabalığını ortadan kaldıran reaktif geçiş sınıfları.

Yerleşik `FooTransition` Ailesi

Flutter, en sık kullanılan dönüşümler için `AnimatedWidget` 'tan türetilmiş hazır açık animasyon widget'ları sunar:

- `FadeTransition(opacity: animation, child: ...)`
- `SlideTransition(position: offsetAnimation, child: ...)`
- `ScaleTransition(scale: animation, child: ...)`
- `RotationTransition(turns: animation, child: ...)`
- `SizeTransition(sizeFactor: animation, child: ...)`

Ne Zaman `AnimatedWidget` Türetilir?

Yeniden kullanılabilir, bağımsız bir animasyonlu bileşen tasarlarken `StatefulWidget` + `addListener` + `setState` yazmak yerine doğrudan `AnimatedWidget` sınıfından kalıtım alınır ve `listenable` parametresine `Animation` nesnesi verilir.

animated_logo_widget.dart (AnimatedWidget Alt Sınıfı)

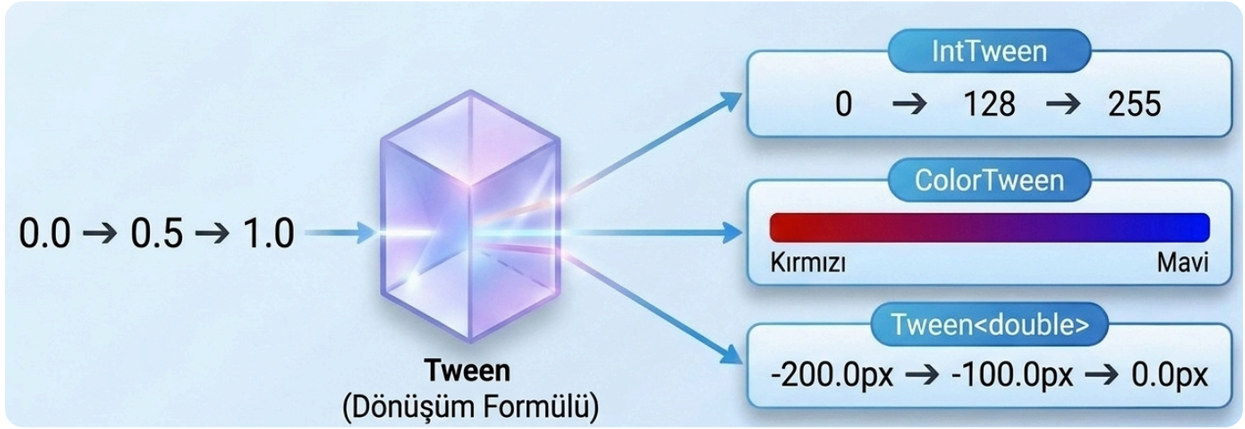
```dart
class AnimatedLogo extends AnimatedWidget {
  const AnimatedLogo({
    super.key,
    required Animation<double> animation,
  }) : super(listenable: animation); // 1. Dinleyiciyi üst sınıfa bağla

  // 2. listenable nesnesini güvenli biçimde Animation<double>'a dönüştür
  Animation<double> get _sizeAnim => listenable as Animation<double>;

  @override
  Widget build(BuildContext context) {
    // 3. Her karede yalnızca bu widget yeniden çizilir!
    return Center(
      child: Container(
        margin: const EdgeInsets.symmetric(vertical: 10),
        height: _sizeAnim.value,
        width: _sizeAnim.value,
        child: const FlutterLogo(),
      ),
    );
  }
}
```

Tween<T>: Veri Dönüştürücü Prizma

In-betweening (ara değer üretimi): 0.0 – 1.0 arasındaki boyutsuz zamanı hedef veri tipine haritalamak.



Doğrusal İnterpolasyon: `lerp(double t)`

Her `Tween<T>` sınıfı, `t ∈ [0.0, 1.0]` ilerleme değeri için başlangıç ve bitiş değerleri arasında matematiksel dönüşüm yapar:

```
değer(t) = begin + t × (end - begin)
```

Özel alt sınıflar: `ColorTween`, `IntTween`, `RectTween`, `AlignmentTween`, `BorderRadiusTween`, `DecorationTween`.

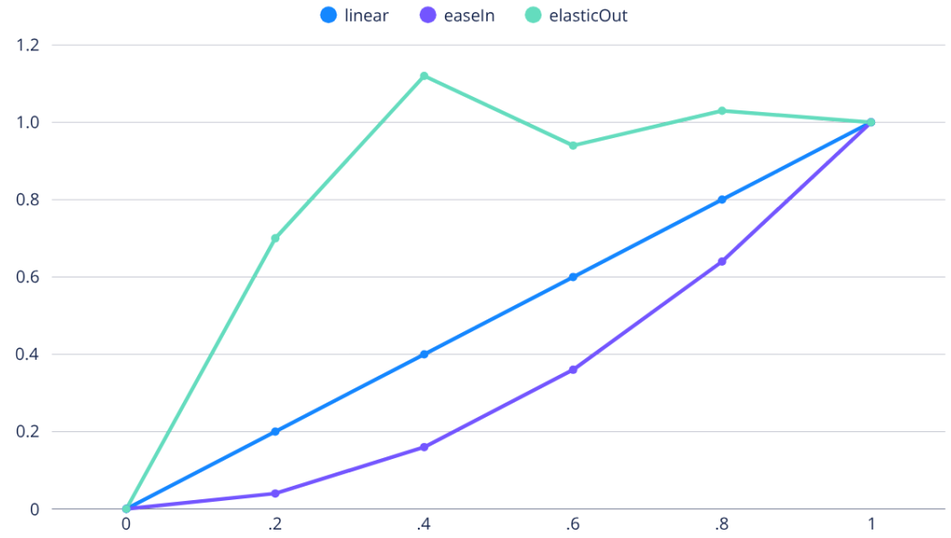
tween_binding_methods.dart

```dart
// Yöntem 1: tween.animate(parent) ile yeni Animation<T> üretme
final Animation<Color?> colorAnim = ColorTween(
  begin: Colors.red,
  end: Colors.blue,
).animate(_controller);

// Yöntem 2: controller.drive(tween) + CurveTween zincirleme
final Animation<Offset> offsetAnim = _controller.drive(
  Tween<Offset>(begin: const Offset(0, 1), end: Offset.zero)
      .chain(CurveTween(curve: Curves.easeOutCubic)),
);
```

Eğriler (Curves & CurvedAnimation): Zamanı Bükmek

Gerçek dünyadaki nesneler sabit hızla hareket etmez; hızlanır, yavaşlar veya yaylanarak durur.



curved_animation_setup.dart

```dart
// İleri ve geri yön için farklı ivme eğrisi tanımlama:
final curvedAnim = CurvedAnimation(
  parent: _controller,
  curve: Curves.easeOutBack, // Hedefe varırken hafif taşma
  reverseCurve: Curves.easeIn, // Kapanırken hızlı ivmelenme
);
```

`Curves.easeOut`

Hızlı başlar, hedefe yaklaşırken yumuşakça fren yapar. Ekrana **giren** UI öğeleri için en doğal seçimdir.

`Curves.easeIn`

Yavaş başlar, hızlanarak çıkar. Ekrandan **ayrılan / kapanan** öğeler için tercih edilir.

`Curves.easeInOut`

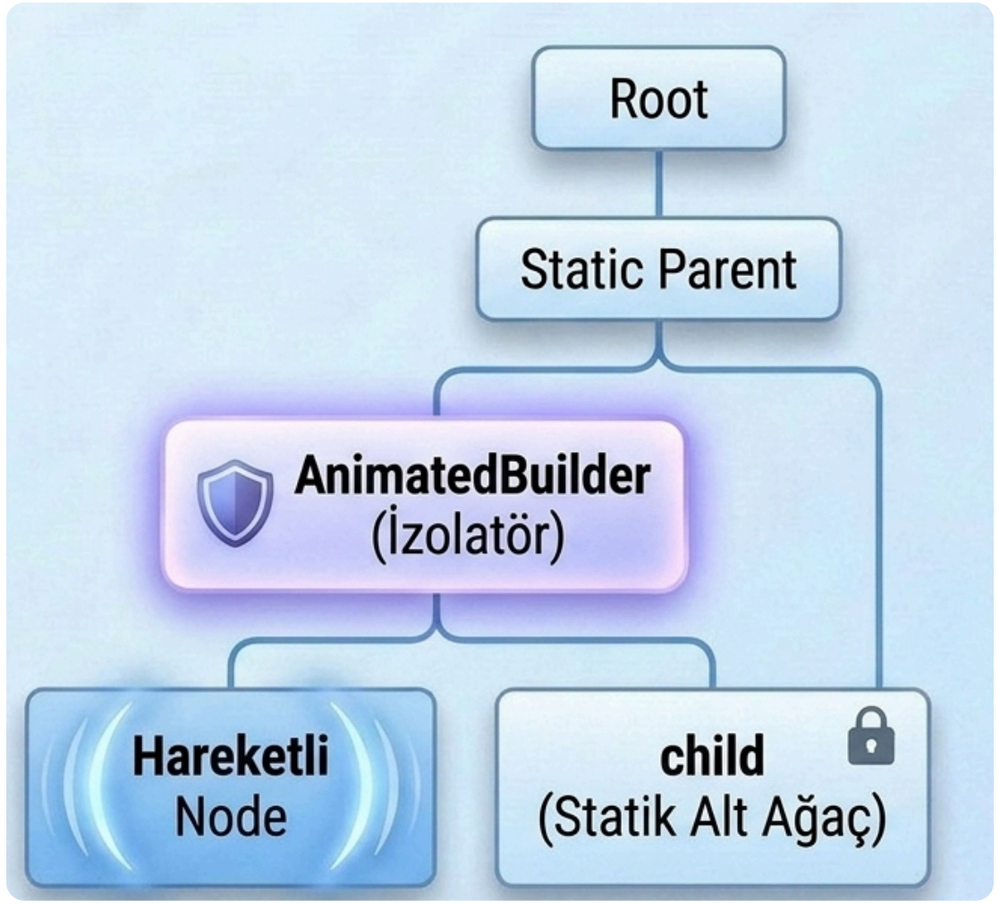
Yavaş kalkış ve yavaş duruş sağlar. Ekran **içinde yer değiştiren** bileşenlerde standarttır.

`bounceOut` / `elasticOut`

Hedefe çarparak sekme veya elastik salınım üretir; oyunsu / dikkat çekici geri bildirim sağlar.

AnimatedBuilder: Widget Ağacında Yeniden Çizimi İzole Etmek

Animasyon mantığını görsel bileşenden ayırmak ve sabit alt ağacı (child) yeniden oluşturmaktan korumak.



animated_builder_isolation.dart

```dart
AnimatedBuilder(
  animation: _controller,
  // Sabit alt ağaç: Animasyon boyunca SADECE 1 KEZ oluşturulur!
  child: const HeavyProfileCard(userName: 'TeamAnimators'),
  builder: (BuildContext context, Widget? cachedChild) {
    // Saniyede 60-120 kez çalışan hafif sarmalayıcı (Transform):
    return Transform.scale(
      scale: 1.0 + (_controller.value * 0.15),
      child: cachedChild, // Ağır widget tekrar build edilmez!
    );
  },
)
```

Yanlış Yaklaşım: Tüm Sayfada `setState()`

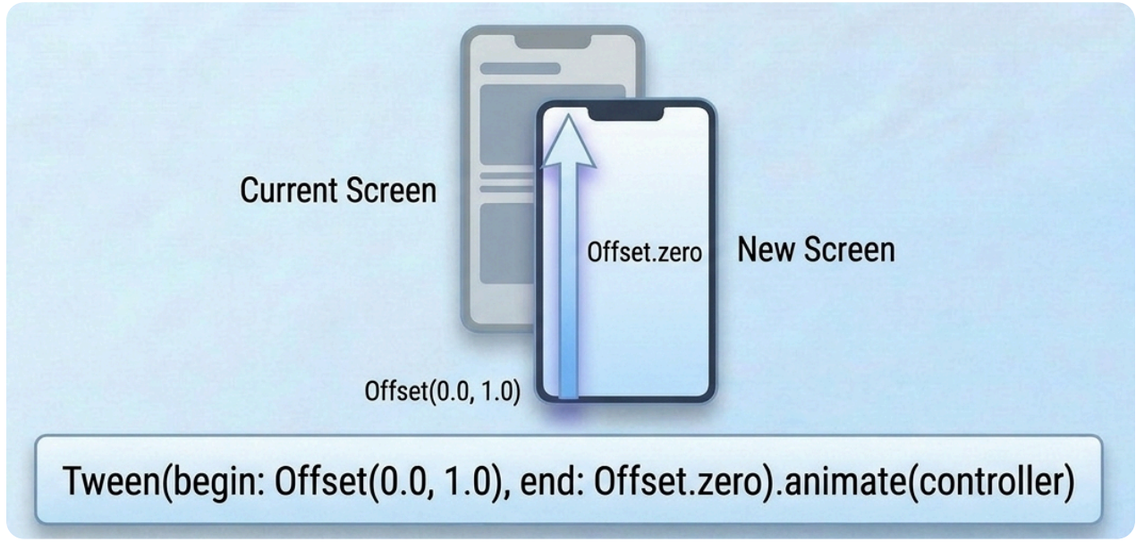
Eğer `_controller.addListener(() => setState(() {}))` kök `Scaffold` içinde çağrılırsa, her karede AppBar, listeler ve tüm statik bileşenler gereksiz yere yeniden oluşturulur.

Doğru Yaklaşım: `AnimatedBuilder` + `child`

`AnimatedBuilder` (Single Responsibility ilkesine uygun olarak) yalnızca geçiş sarmalayıcısını yeniden çizer; `child` parametresindeki statik alt ağacı kilitli tutar.

Sayfa Geçişleri: PageRouteBuilder ve SlideTransition

Standart MaterialPageRoute yerine iki ekran arasında kesirli koordinatlarla (Offset) özel geçiş tasarlamak.



custom_page_route_transition.dart

```dart
Route _createSlideUpRoute() {
  return PageRouteBuilder(
    pageBuilder: (context, animation, secondaryAnimation) => const Page2(),
    transitionsBuilder: (context, animation, secondaryAnimation, child) {
      // Offset(0.0, 1.0) = Ekranın tam 1 boy aşağısı (Alt kenar)
      // Offset.zero      = Offset(0.0, 0.0) (Normal ekran konumu)
      const begin = Offset(0.0, 1.0);
      const end = Offset.zero;
      const curve = Curves.easeOutCubic;

      final tween = Tween(begin: begin, end: end)
          .chain(CurveTween(curve: curve));

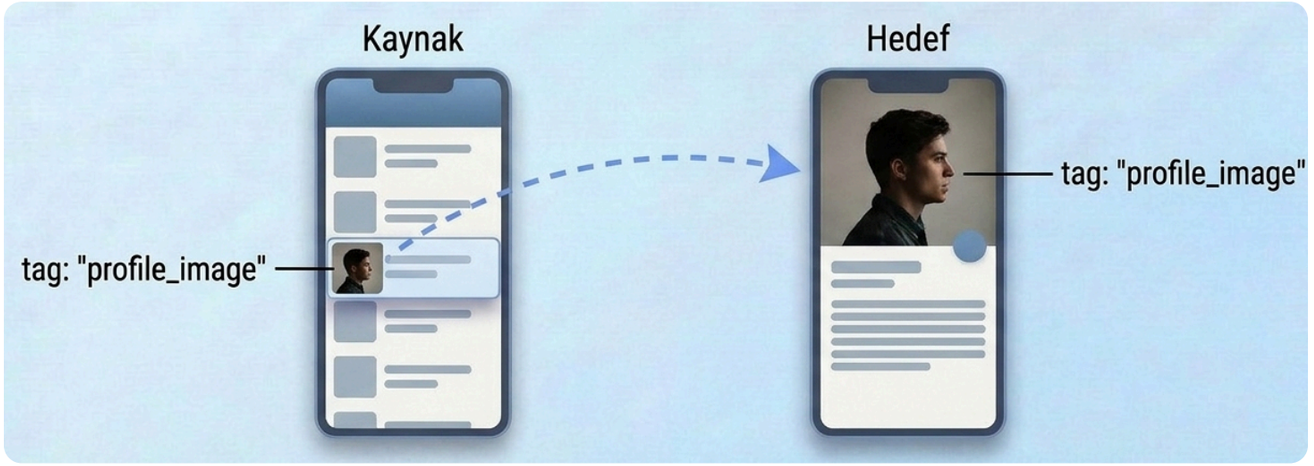
      return SlideTransition(
        position: animation.drive(tween),
        child: child,
      );
    },
  );
}
```

Kesirli (Fractional) `Offset(dx, dy)` Koordinat Sistemi

`SlideTransition` piksel değil, ekran boyutunun katlarını kullanır: `Offset(0.0, 1.0)` yeni sayfayı ekranın **altından yukarı** kaydırır; `Offset(1.0, 0.0)` ise **sağdan sola** kaydırır. `Navigator.of(context).push(_createSlideUpRoute())` ile çağrılır.

Hero Geçişleri: Uzayda ve Ekranlar Arasında Süreklilik

Kullanıcı bir listedeki öğeye tıkladığında, öğenin detay sayfasına uçarak yerleşmesini sağlayan desen.



1. Kaynak Rota (Liste Ekranı)

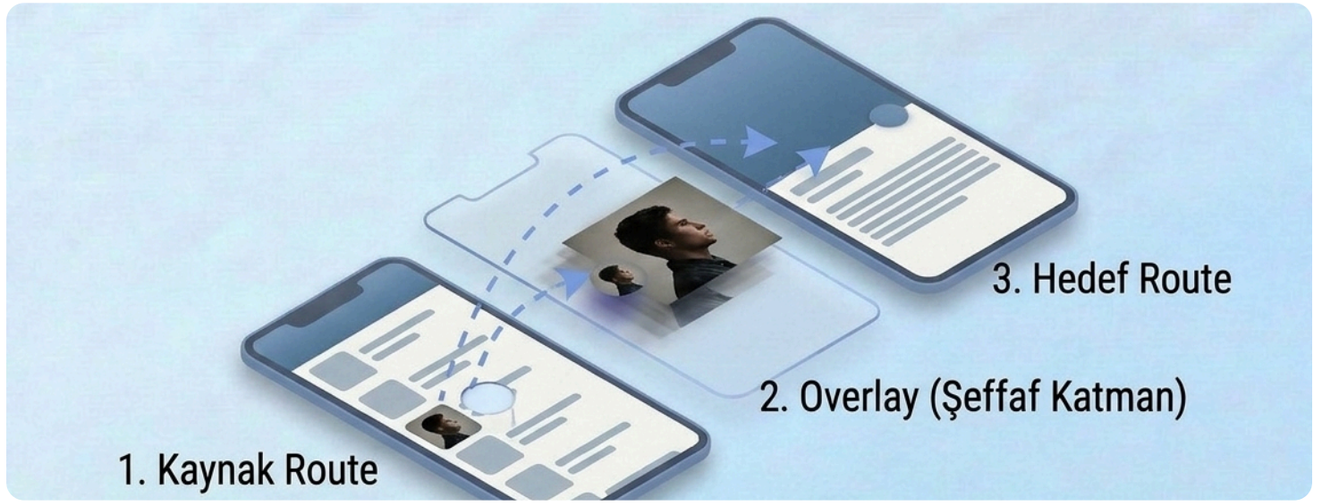
```dart
GestureDetector(
  onTap: () => Navigator.push(context, detailRoute()),
  child: Hero(
    tag: 'profile_image', // Benzersiz ortak kimlik
    child: ClipOval(
      child: Image.asset('assets/avatar.jpg', width: 56),
    ),
  ),
)
```

2. Hedef Rota (Detay Ekranı)

```dart
Scaffold(
  body: Center(
    child: Hero(
      tag: 'profile_image', // Kaynak ile birebir aynı tag!
      child: Image.asset('assets/avatar.jpg', width: 280),
    ),
  ),
)
```

Hero Animasyonlarının Sahne Arkası: Overlay Katmanı

Bir widget iki farklı sayfa ağacında aynı anda bulunamaz; peki ekranda nasıl kesintisiz uçar?



1 Kalkış (`t = 0.0`) ve Boyut Hesabı

Flutter, kaynak ve hedef `Hero`'ların ekran koordinatlarını (`Rect`) hesaplar. Hedef widget geçici olarak `Offstage` ile gizlenir.

2 `Overlay` Katmanında Kavisli Uçuş

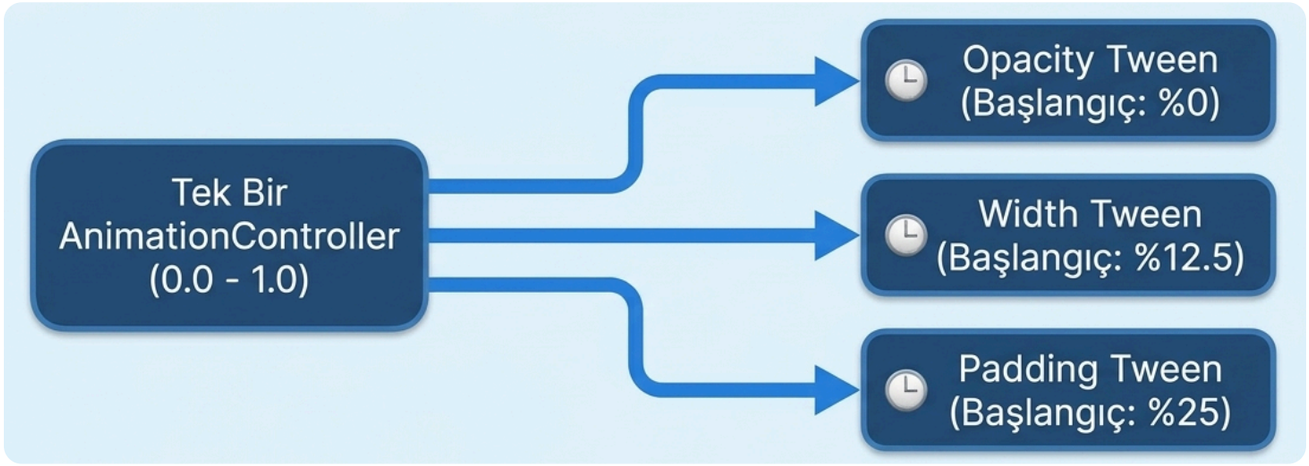
Hero widget'ı her iki rotanın da üstünde yer alan şeffaf `Overlay` katmanına taşınır. `MaterialRectArcTween` sayesinde düz çizgi yerine Material Design standartlarında **kavisli bir yay** çizerek uçar.

3 İniş (`t = 1.0`) ve Radyal Dönüşüm

Uçuş bittiğinde widget `Overlay`'den kaldırılır ve hedef rotadaki gerçek yerine yerleşir. Yuvarlaktan kareye dönüşümlerde `RadialExpansion` (`ClipOval` + `ClipRect`) kullanılır.

Koreografi: Tek Zamanlayıcı ile Çoklu Özellik Orkestrasyonu

Ardışık veya örtüşen animasyonlarda birden fazla Controller yerine tek bir Controller kullanmak.



1 Tek `AnimationController`

Tüm koreografi `0.0` 'dan `1.0` 'a ilerleyen **tek bir ana saat** tarafından yönetilir. Böylece animasyonlar arasında senkron kayması yaşanması imkânsız hale gelir.

2 Bağımsız `Tween` Nesneleri

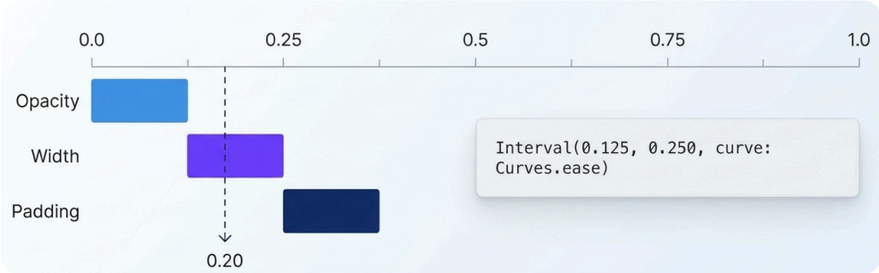
Anime edilecek her görsel özellik (`opacity` , `width` , `height` , `padding` , `borderRadius` , `color`) kendi `Tween` nesnesine bağlanır.

3 Tek Komutla İleri / Geri

Tek bir `_controller.forward()` veya `_controller.reverse()` çağrısıyla 6 farklı özelliğin tamamı kusursuz sıra ve zamanlamayla oynatılır veya geri sarılır.

Zamanı Dilimlemek: Interval Sınıfı

Ana zaman çizelgesinin (0.0 – 1.0) yalnızca belirli bir yüzdelik diliminde aktif olan alt animasyonlar.



staggered_interval_definitions.dart

```dart
// 1. Opaklık: Zamanın ilk %15'lik diliminde (0.000 -> 0.150) tamamlanır
opacity = Tween<double>(begin: 0.0, end: 1.0).animate(
  CurvedAnimation(parent: controller, curve: const Interval(0.0, 0.150, curve: Curves.ease)),
);
// 2. Genişlik: %12.5 ile %25.0 arasında (0.125 -> 0.250) büyür
width = Tween<double>(begin: 50.0, end: 150.0).animate(
  CurvedAnimation(parent: controller, curve: const Interval(0.125, 0.250, curve: Curves.ease)),
);
// 3. Padding: %25.0 ile %37.5 arasında (0.250 -> 0.375) yukarı kayar
padding = EdgeInsetsTween(
  begin: const EdgeInsets.only(bottom: 16),
  end: const EdgeInsets.only(bottom: 75),
).animate(CurvedAnimation(
  parent: controller,
  curve: const Interval(0.250, 0.375, curve: Curves.ease),
));
```

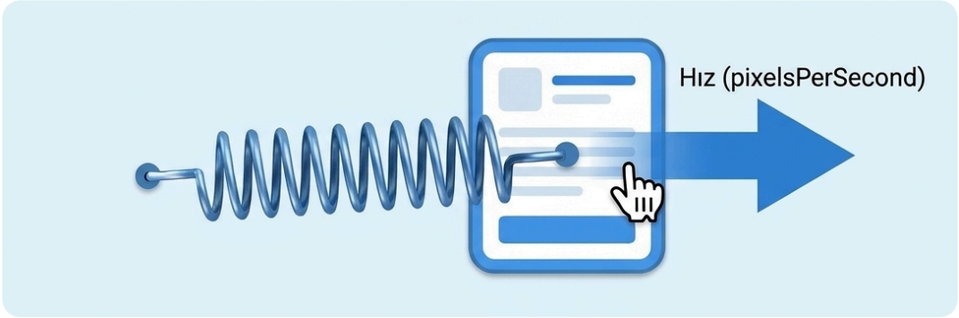
`Interval(begin, end)` Matematiği

`Interval` sınıfı `Curve`'den türetilmiştir:

- Ana controller değeri `begin`'den küçükken çıkış `0.0`'dır.
- `begin` ile `end` arasındaki dilimi kendi içinde `0.0 → 1.0` aralığına normalize eder.
- Ana controller `end` değerini geçtiğinde çıkış `1.0`'da sabitlenir.

Fizik Simülasyonları: SpringSimulation ve Sürükleme Hızı

Kullanıcı bir kartı sürükleyip bıraktığında, sabit süre yerine parmak fırlatma hızını (velocity) koruyan yay fiziği.



physics_card_drag_spring.dart (flutter/physics.dart)

```dart
void _runSpringAnimation(Offset pixelsPerSecond, Size size) {
  _animation = _controller.drive(
    AlignmentTween(begin: _dragAlignment, end: Alignment.center),
  );
  // 1. Piksel/saniye hızını (GestureDetector onPanEnd) birim hıza (0.0-1.0) çevir
  final unitsPerSecond = Offset(pixelsPerSecond.dx / size.width, pixelsPerSecond.dy / size.height);
  final unitVelocity = unitsPerSecond.distance;

  // 2. Yay fiziği parametreleri: Kütle (mass), Sertlik (stiffness), Sönümleme (damping)
  const spring = SpringDescription(mass: 30, stiffness: 1, damping: 1);
  final simulation = SpringSimulation(spring, 0, 1, -unitVelocity);

  // 3. Controller'ı sabit Duration ile değil, fizik simülasyonuyla sür!
  _controller.animateWith(simulation);
}
```

`SpringDescription` Parametreleri

- `mass` **(Kütle):** Nesnenin eylemsizliği (ataleti).
- `stiffness` **(Sertlik):** Yay sabiti; merkeze çekim gücü.
- `damping` **(Sönümleme):** Sürtünme; salınımın ne kadar sürede duracağını belirler.

Neden `animateWith()` ?

Standart `forward()` sabit bir süre ister; `animateWith(simulation)` ise yay enerjisi sönümlenene kadar doğal sürede çalışır.

Flutter Animasyon Mimarisi: Büyük Resim

Donanım VSync sinyalinden 120 FPS (8.3 ms) kare bütçesiyle piksellere uzanan 3 katmanlı uçtan uca akış.

1 KATMAN 1: ZAMANLAMA VE MOTOR (TIMING & ENGINE LAYER)

KARE BÜTÇESİ: 16.6 MS (60HZ) / 8.3 MS (120HZ)

Ticker (`SchedulerBinding`)
Donanım VSync darbesiyle her karede zaman farkını (`Elapsed Duration`) üretir.

`AnimationController`
Süreyi `0.0 ➔ 1.0` aralığına çevirir; `forward` , `reverse` , `repeat` komutlarını yürütür.

`Simulation` (`flutter/physics.dart`)
`SpringSimulation` / `GravitySimulation` ile hız ve kuvvete dayalı konum üretir.

2 KATMAN 2: EĞRİ VE VERİ HARİTALAMA (CURVE & TWEEN LAYER)

DEĞER DÖNÜŞÜMÜ: ANİMATİON<DOUBLE> ➔ ANİMATİON<T>

`CurvedAnimation` & `Curves`
Doğrusal ilerlemeyi `easeOut` , `bounceOut` , `Cubic` fonksiyonlarıyla şekillendirir.

`Interval(begin, end)`
Kademeli (Staggered) koreografilerde zaman çizelgesini alt pencerelere böler.

`Tween<T>` (`.animate` / `.drive`)
`ColorTween` , `RectTween` , `Tween<Offset>` ile UI veri tiplerini üretir.

3 KATMAN 3: GÖRÜNÜM VE İZOLASYON (RENDER & WIDGET LAYER)

HEDEF: MİNİMUM BUİLD & GPU KATMAN OPTİMİZASYONU

`AnimatedFoo` (Örtük)
`AnimatedContainer` , `TweenAnimationBuilder`

`FooTransition` (Açık)
`FadeTransition` , `SlideTransition` , `AnimatedWidget`

`AnimatedBuilder`
Statik `child` önbellekleme ile izole yeniden çizim

`Hero` & `PageRouteBuilder`
`Overlay` uçuşu ve ekranlar arası rota geçişleri

Örtük vs. Açık Animasyonlar ve 4 Altın Performans Kuralı

Mimari karar matrisi ve üretim ortamında 60–120 FPS akıcılığı koruma ilkeleri.

Kriter	Örtük (Implicit) Animasyonlar (`AnimatedFoo` / `TweenAnimationBuilder`)	Açık (Explicit) Animasyonlar (`AnimationController` + `AnimatedBuilder`)
Kod & Yaşam Döngüsü	Çok düşük; `Controller`, `vsync` veya `dispose()` yönetimi gerekmez.	Manuel kurulum; `SingleTickerProviderStateMixin` ve `dispose()` zorunludur.
Zaman Çizelgesi Kontrolü	Tek yönlü otopilot: Eski değerden yeni değere bir kez kayar ve durur.	Tam kontrol: `forward()`, `reverse()`, `repeat()`, `stop()`, `animateWith()`.
Çoklu Koreografi	Bağımsız basit durum geçişleri için uygundur.	`Interval` ile kademeli (Staggered) ve senkronize çoklu hareketler için idealdir.

01 Build Alanını Küçült

`AnimatedBuilder` ve `TweenAnimationBuilder` içinde değişmeyen ağır bileşenleri mutlaka `child` parametresiyle dışarıdan verin.

02 Controller'ı Dispose Et

Açık animasyonlarda `dispose()` içinde `_controller.dispose()` çağrısını asla unutmayın; aksi halde arka planda `Ticker` sızıntısı oluşur.

03 GPU Dostu Widget Seç

Animasyon sırasında `Opacity` widget'ını her karede yeniden çizmek yerine `FadeTransition` veya `AnimatedOpacity` kullanın; karmaşık alanları `RepaintBoundary` ile ayırın.

04 Anlamlı & Erişilebilir Ol

Standart UI geçişlerini **200–350 ms** aralığında tutun.
`MediaQuery.disableAnimationsOf(context)` ile kullanıcı erişilebilirlik tercihlerine saygı duyun.

"Hareket, anlam taşıdığında değerlidir."

Flutter'ın zengin animasyon mimarisi; basit bir renk değişiminden çok adımlı koreografilere ve yay fiziği simülasyonlarına kadar her ölçekte 60–120 FPS akıcılık sunar.

İNCELENEN 8 RESMİ DOKÜMAN

1. Introduction & Overview
2. Animation Tutorial (`AnimatedWidget` / `Builder`)
3. Implicit Animations (`AnimatedFoo`)
4. Hero & Radial Hero Animations
5. Staggered Animations (`Interval`)
6. Page Route Transitions (`PageRouteBuilder`)
7. Physics Card Drag (`SpringSimulation`)
8. AnimatedContainer & Opacity Cookbook

6. TAKIM: TEAMANİMATORS

- **Aydın Yiğit Oğuz**
- **Arman Dündar**
- **Sefa Omaç**
- **Yusuf Cemal Durmuş**

Sivas Cumhuriyet Üniversitesi · Şarkışla UBYO
Mobil Programlama I · 2026 Güz Dönemi

